GLOBAL SOLUTION

Disciplina: Cloud Security

CyberIncident API

Plataforma de Gerenciamento de Incidentes de Segurança Cibernética

Aluno: Breno Aveiro

Data: 02/11/2026

1. Introdução

Incidentes como phishing, malware, acessos indevidos e vazamentos exigem registro, classificação e acompanhamento até a resolução. Em muitas organizações, esse trabalho é realizado pelo Centro de Operações de Segurança (SOC, Security Operations Center).

A CyberIncident API simula o gerenciamento de incidentes de um SOC. Permite cadastrar, consultar, classificar e acompanhar ocorrências fictícias, sem realizar detecção ou monitoramento real. O projeto demonstra o desenvolvimento e a hospedagem de uma API REST com SQLite na Azure. Código: https://github.com/brenoaveiro/api-tarefas-fastapi-sqlite-azure. Aplicação: https://20.80.10.214/.

A aplicação evoluiu de uma API de tarefas. Foram mantidos a VM, o disco e o banco SQLite, enquanto a interface, os endpoints e os modelos passaram a representar incidentes. A tabela antiga foi preservada.

2. Objetivos

O objetivo é desenvolver, testar e hospedar na Azure uma API REST com SQLite. Para isso, o projeto busca:

- registrar e classificar incidentes por categoria, severidade, status, origem e ativo afetado;
- implementar CRUD, filtros e estatísticas calculadas a partir do banco;
- oferecer dashboard e Swagger com identidade visual própria;
- hospedar a API em VM Azure com Nginx, systemd e armazenamento persistente;
- aplicar controles de rede, restrição de escrita e HTTPS;
- validar funcionalidades com testes locais, produção e integração contínua;
- documentar arquitetura, custos, resultados e evidências reais.

3. Tecnologias utilizadas

Python 3.12 é a linguagem utilizada na VM. Sua sintaxe legível e suas bibliotecas facilitam a implementação da lógica, da validação e do acesso ao banco.

FastAPI recebe as requisições HTTP, executa as rotas e gera a especificação OpenAPI. Foi escolhido pela integração entre tipos Python, validação de dados e documentação automática.

Pydantic define e valida os dados de entrada e saída. Restringe severidades e categorias, valida endereços IP e devolve HTTP 422 para entradas inválidas.

SQLAlchemy é o ORM que relaciona objetos Python a tabelas SQL. As consultas parametrizadas reduzem o risco de injeção SQL e organizam a persistência.

SQLite armazena tabelas e registros em um arquivo local, dispensando servidor de banco separado.

Uvicorn executa o FastAPI como servidor ASGI e escuta somente em 127.0.0.1:8000 na VM.

Nginx recebe o tráfego público nas portas 80 e 443, encaminha as requisições ao Uvicorn, gerencia HTTPS e restringe operações de escrita por IP.

systemd inicia o serviço com a VM, reinicia-o em caso de falha e aplica restrições de execução.

HTML, CSS e JavaScript compõem o dashboard e o tema do Swagger sem frameworks pesados, economizando memória na VM de 1 GiB.

Swagger UI oferece documentação interativa com tema próprio e arquivos locais. Git/GitHub versionam o código; GitHub Actions executa a integração contínua; Pytest testa a aplicação e Mermaid gera os diagramas.

As dependências principais estão fixadas em requirements.txt: FastAPI 0.162.0, Starlette 1.10.0, Pydantic 2.14.2, SQLAlchemy 2.0.55 e Uvicorn 0.54.0. A atualização do Starlette corrigiu 14 vulnerabilidades identificadas anteriormente pelo pip-audit; a verificação registrada não apontou vulnerabilidades conhecidas na versão final.

4. Arquitetura

A requisição percorre navegador, IP público, regras do NSG, rede virtual, Nginx, Uvicorn/FastAPI e SQLite. O Nginx recebe HTTP/HTTPS e encaminha ao Uvicorn local. O FastAPI valida a entrada, aplica as regras de negócio e acessa o banco pelo SQLAlchemy.

O Uvicorn não é exposto à internet; o Nginx centraliza TLS e regras de acesso. O SQLite permanece no disco gerenciado mesmo após reinícios. Os Diagramas 1 e 2 ilustram essa arquitetura.

5. API REST

A API REST organiza recursos em rotas e utiliza JSON. GET consulta; POST cria; PUT atualiza os dados do recurso; PATCH altera campos específicos; DELETE exclui. Os principais códigos são 200 (sucesso), 201 (criado), 204 (sem corpo), 404 (não encontrado), 422 (entrada inválida) e 500 (erro interno).

Método e rota	Função	Códigos
GET /	Dashboard (página HTML)	200
GET /health	Verifica a API e a comunicação com o SQLite	200, 503
GET /incidentes	Lista com paginação e filtros por severidade, status e categoria	200, 422
GET /incidentes/estatisticas	Contagens reais por status, severidade e categoria	200
GET /incidentes/{id}	Consulta um incidente	200, 404
POST /incidentes	Cadastra um incidente (status inicial: aberto)	201, 422
PUT /incidentes/{id}	Atualiza todos os campos	200, 404, 422
PATCH /incidentes/{id}/status	Altera somente o status	200, 404, 422
DELETE /incidentes/{id}	Exclui um incidente de demonstração	204, 404

Validações. Pydantic verifica tamanho dos textos, valores de categoria e severidade e IPs IPv4/IPv6; o SQLite reforça valores permitidos por restrições CHECK.

Regras de negócio. O fluxo de tratamento é aberto, em análise, contido e resolvido. Transições inválidas retornam 422. A resolução preenche resolvido_em em UTC, e a reabertura limpa esse campo.

Estatísticas. /incidentes/estatisticas consulta o SQLite com GROUP BY e informa totais por categoria, severidade e status, incluindo categorias zeradas. O dashboard utiliza esses dados reais.

Erros. Falhas internas recebem resposta genérica HTTP 500; falhas do banco no health check retornam 503.

Interface. O dashboard (GET /) apresenta cartões de indicadores, gráfico de severidade e incidentes recentes em tema SOC responsivo. Os textos são inseridos com textContent e protegidos por CSP para reduzir riscos de XSS. O Swagger (/docs) usa tema personalizado e recursos locais, mantendo testes interativos.

6. Banco de dados SQLite

O banco SQLite fica em /var/lib/api-tarefas/tarefas.db. Contém a tabela incidentes e a tabela legada tarefas, preservada. Incidentes registra ID, título, descrição, categoria, severidade, status, IP de origem, ativo, responsável e datas de criação, atualização e resolução (UTC). Índices favorecem filtros; restrições CHECK reforçam a integridade (Diagrama 3).

O modo WAL (write-ahead logging) reduz disputas entre leitura e escrita. O arquivo pertence ao usuário apitarefas e tem permissão 600.

Persistência. O SQLite está no disco gerenciado, não na memória do serviço. Um incidente permaneceu disponível após reiniciar a aplicação, como mostra a Figura 16.

Migração e backup. A inicialização cria somente tabelas ausentes, sem apagar dados. Antes de cada deploy, o script realiza e valida um backup consistente do SQLite; se falhar, a atualização é cancelada. A existência das tabelas é conferida em sqlite_master.

Escolha e limites. O SQLite é simples, não exige serviço adicional nem custo específico e atende ao pequeno volume da API. Para alta concorrência ou replicação, seria mais indicado um banco gerenciado.

7. Microsoft Azure

A infraestrutura utiliza Azure for Students, com créditos e restrições de região e cota. Os recursos estão no Resource Group GS_Cloud_API, em North Central US.

Resource Group. Agrupa os recursos da solução e facilita sua administração, controle de acesso e exclusão.

VM e SKU. A GS-Cloud-API-VM usa Ubuntu Server 24.04 LTS e SKU Standard_B2ats_v2, da série B (burstable), apropriada a cargas leves com picos ocasionais. Possui 2 vCPUs e 1 GiB de RAM; foi configurado swap de 1 GiB para aliviar a memória limitada. A SKU determina capacidade e custo da VM.

Escolha da SKU. A B1s inicialmente prevista não estava disponível nas condições da assinatura. Políticas regionais e cotas impediram alternativas no Brasil; a B2ats_v2 em North Central US foi a opção econômica viável.

Disco gerenciado. O GS_Cloud_API_Disk é um Standard SSD LRS de 32 GiB, administrado pela Azure. A redundância local mantém cópias no mesmo datacenter; nele ficam Ubuntu, código e SQLite, garantindo persistência.

Rede. A VNet GS_Cloud_API_VNet (10.10.0.0/16) contém a subnet GS_Cloud_API_Subnet (10.10.1.0/24), onde a VM usa IP privado 10.10.1.4. A NIC conecta a VM à rede, e o IP público estático é 20.80.10.214.

NSG. O GS_Cloud_API_NSG permite SSH (22) apenas do IP administrativo e HTTP/HTTPS (80/443) da internet. As demais entradas são negadas. A porta interna 8000 não é publicada.

Custos. A VM, o disco e o IP público são cobrados; VNet e NSG não têm custo direto. A estimativa de tabela é de aproximadamente US$ 13/mês com a VM ligada continuamente (VM US$ 6,9; disco US$ 2,6; IP US$ 3,7). Os valores dependem da região e não equivalem ao saldo real da assinatura. Desalocar interrompe a computação, mas não as cobranças de disco e IP. O certificado TLS é gratuito.

8. Implantação

O código é mantido no GitHub. O install.sh configura Python, Nginx, dependências, usuário apitarefas, ambiente virtual e serviço systemd. O deploy.sh faz backup do SQLite, atualiza o Git, instala dependências, sincroniza configurações com validação do Nginx e reinicia o serviço, terminando com health check. Se o backup ou a configuração falhar, a atualização é interrompida ou revertida.

O serviço api-tarefas executa o Uvicorn sem privilégios administrativos, com isolamento pelo systemd e escrita limitada a /var/lib/api-tarefas.

HTTPS. O script deploy/https_ip.sh configurou certificado Let’s Encrypt para o IP público, válido por cerca de seis dias. Utiliza Certbot compatível, desafio HTTP-01 via webroot e renovação automática por timer do systemd, com recarga do Nginx. Foram registrados testes de certificado, cadeia de confiança e renovação.

O projeto mantém HTTP como contingência e não ativa HSTS nem redirecionamento obrigatório, considerando que a VM pode ficar desligada e o certificado de curta duração pode expirar. HTTPS deve ser priorizado no acesso.

O Diagrama 5 resume o deploy; os procedimentos completos e de rollback constam em docs/DEPLOY_AZURE_REAL.md.

9. Segurança

Controles implementados:

- Rede. O NSG libera 80/443 publicamente e SSH somente do IP administrativo. Uvicorn permanece em localhost e o acesso SSH usa chave pública.
- Escrita. GET é público; POST, PUT, PATCH e DELETE são permitidos somente a IPs autorizados no Nginx. Outros clientes recebem HTTP 403, em HTTP e HTTPS.
- HTTPS. TLS 1.2/1.3, certificado Let’s Encrypt válido e renovação automatizada.

- Aplicação. ORM com parâmetros, validações Pydantic, proteção contra XSS no painel, CSP, CORS restrito e erros sem detalhes internos.
- Sistema. Serviço com usuário não privilegiado e isolamento systemd; banco e .env com permissão 600.
- Segredos e dependências. .gitignore impede versionar chaves, .env, bancos e backups. Versões de bibliotecas são fixadas e avaliadas pelo pip-audit.

Autenticação. Não há contas individuais. Para a demonstração com dados fictícios, a escrita foi restrita por IP. Uma evolução adequada exigiria autenticação por token ou provedor de identidade, sempre sobre HTTPS.

Limitações. A escrita depende do IP administrativo; os backups estão no mesmo disco; e há apenas uma VM, sem redundância.

10. Testes

Testes locais. Foram registrados 96 testes automatizados aprovados, sem avisos, com SQLite isolado. Eles abrangem CRUD, filtros, validações, regras de status, estatísticas, persistência, integridade, interface, OpenAPI, assets do Swagger, segurança e configurações de deploy.

Integração contínua. O GitHub Actions instala dependências e executa os testes em ambiente limpo a cada atualização. Não possui credenciais Azure nem executa deploy automático.

Produção. O script testar_producao.sh registrou 58 verificações aprovadas, incluindo HTTPS, dashboard, Swagger, CRUD, estatísticas, códigos 403/404/422, persistência, porta 8000 fechada, certificado e SQLite. Um teste negativo com chave SSH inválida confirmou saída diferente de zero e contabilização de falhas. Os logs permanecem em docs/evidencias.

11. Evidências

As Figuras 1 a 6 registram o Portal Azure; as Figuras 7 a 13 e 17 mostram a aplicação e o GitHub no navegador; as demais apresentam terminal e testes. URLs e horários foram adicionados às capturas de navegador, e dados administrativos sensíveis foram ocultados.

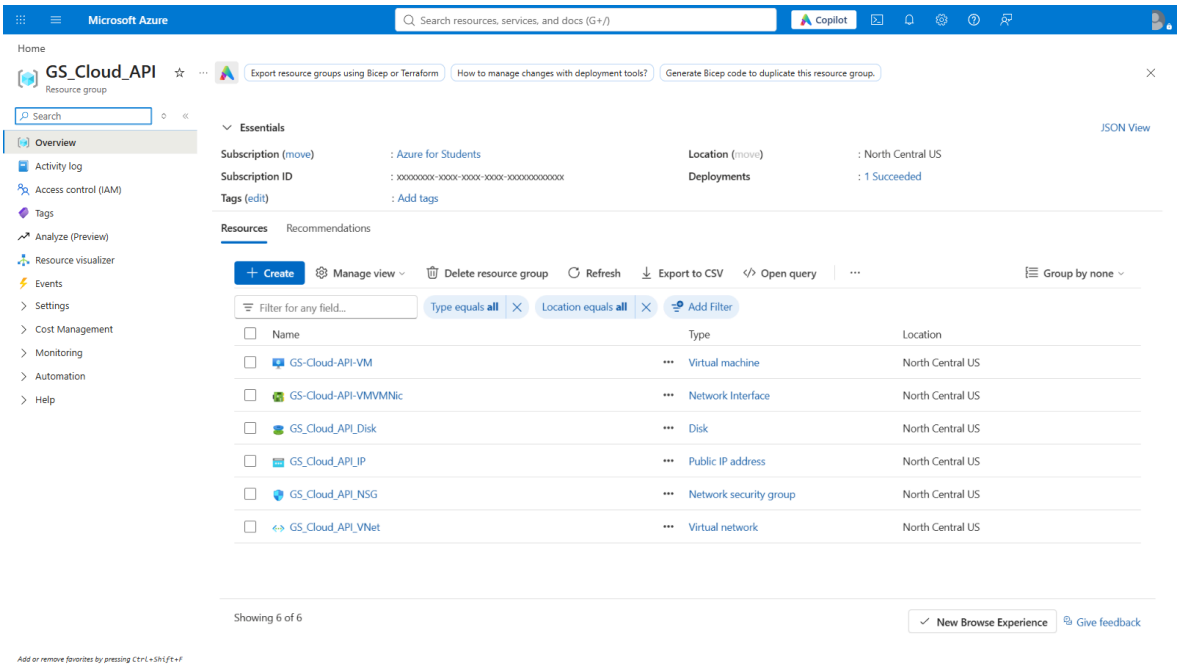


Figura 1 - Resource Group com os recursos da aplicação.

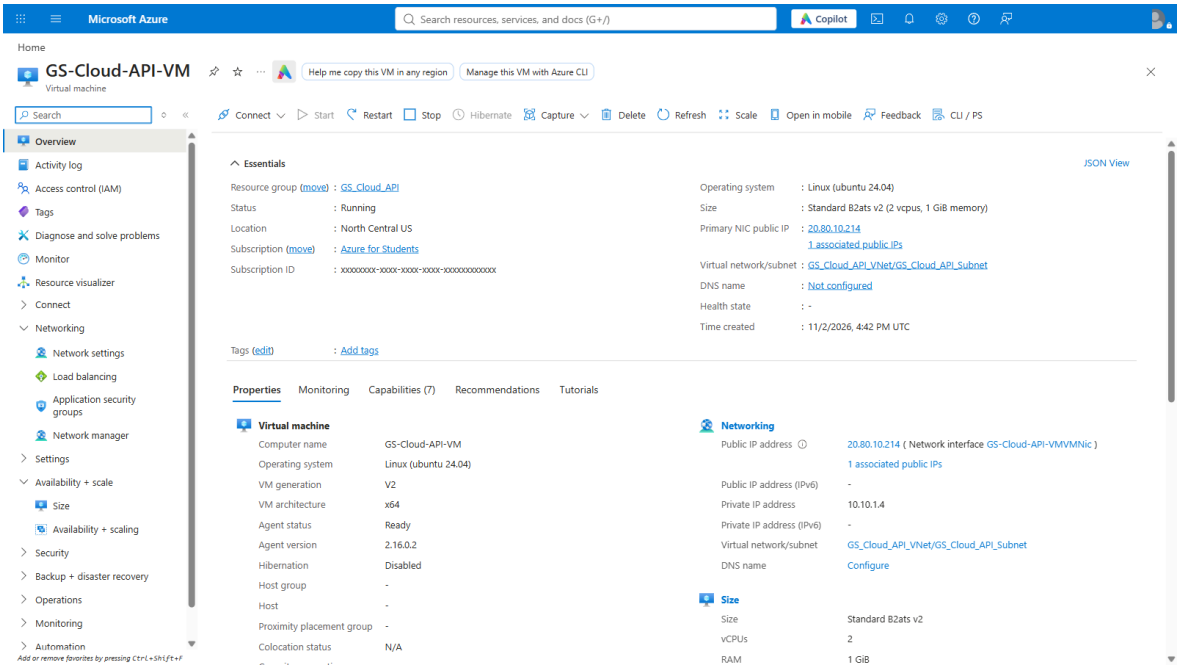


Figura 2 - Visão geral da VM Azure em execução.

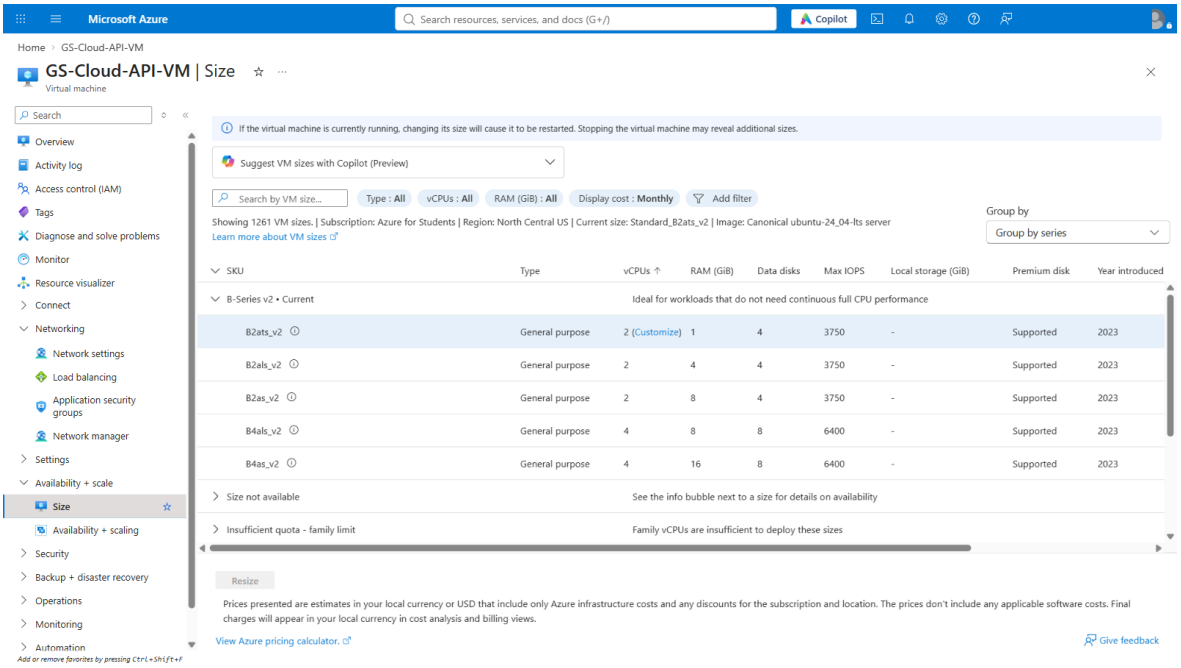


Figura 3 - SKU, processamento e memória da VM.

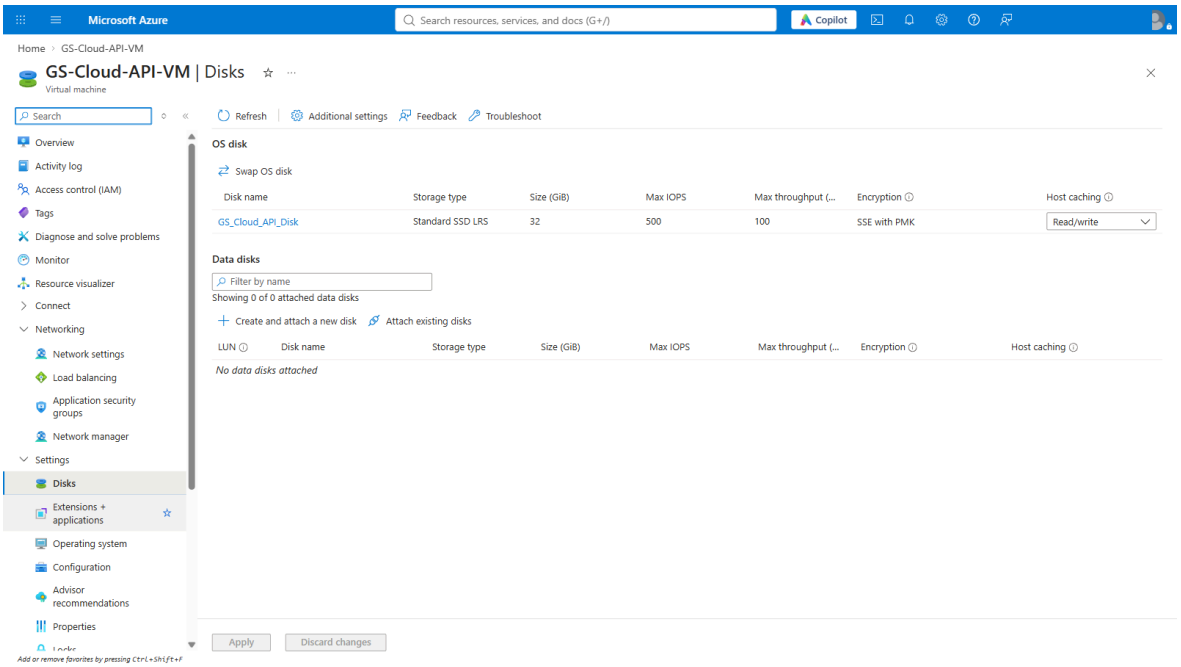


Figura 4 - Disco gerenciado e armazenamento persistente.

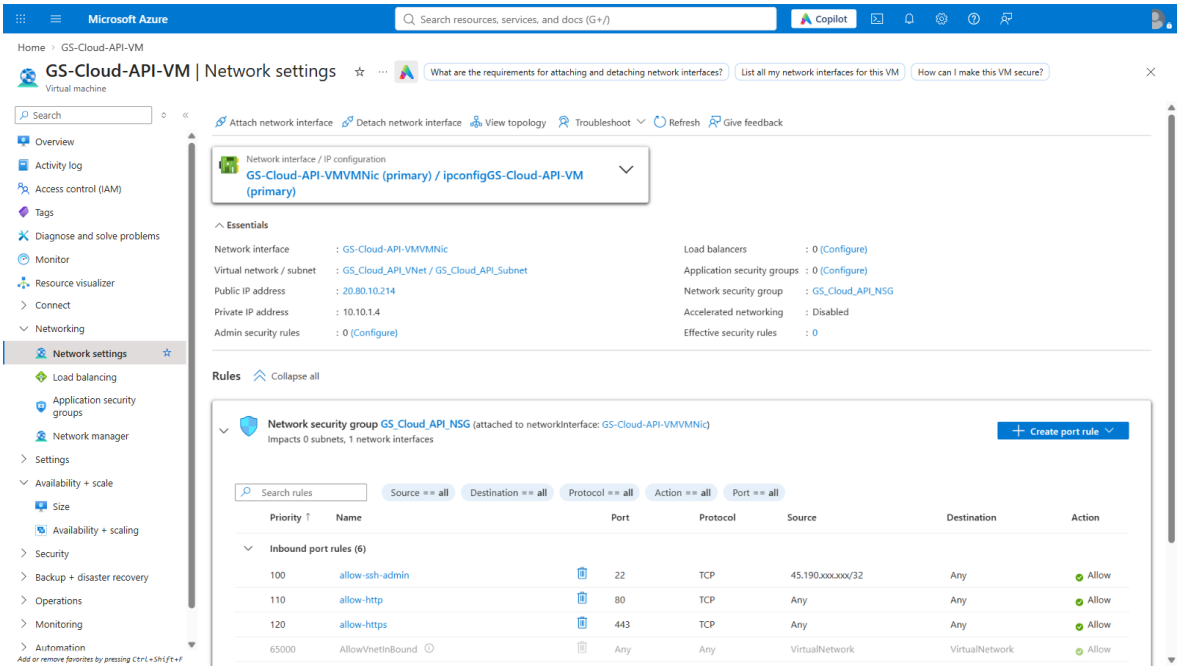


Figura 5 - Rede virtual e interface de rede da VM.

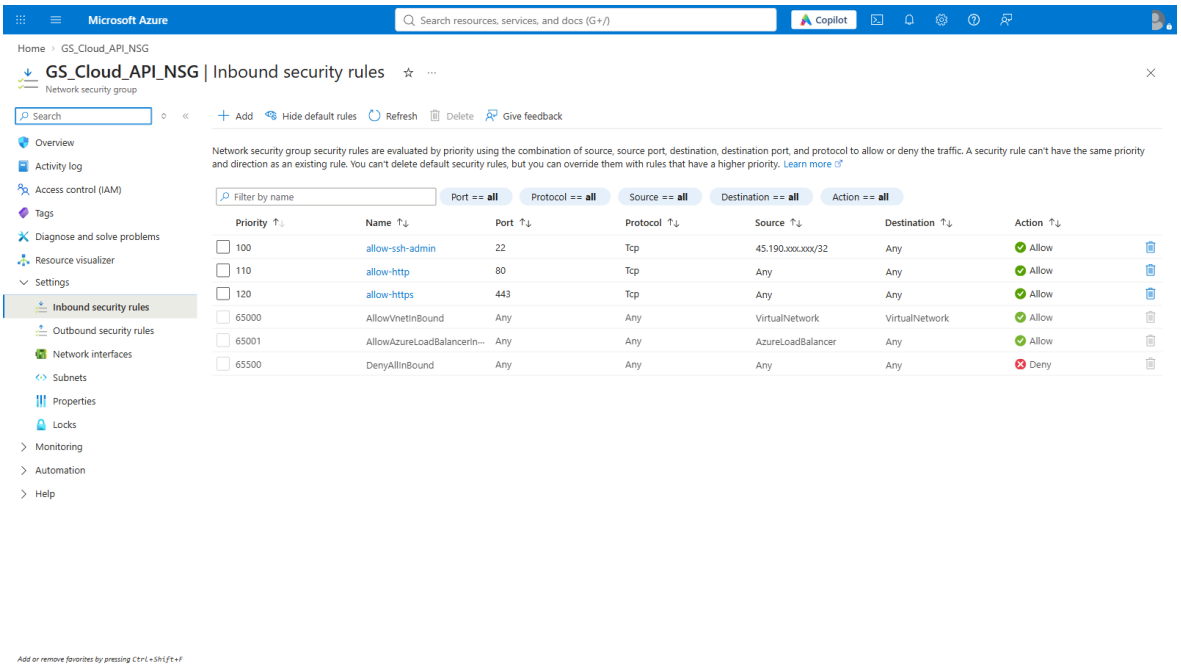


Figura 6 - Regras de entrada do NSG.

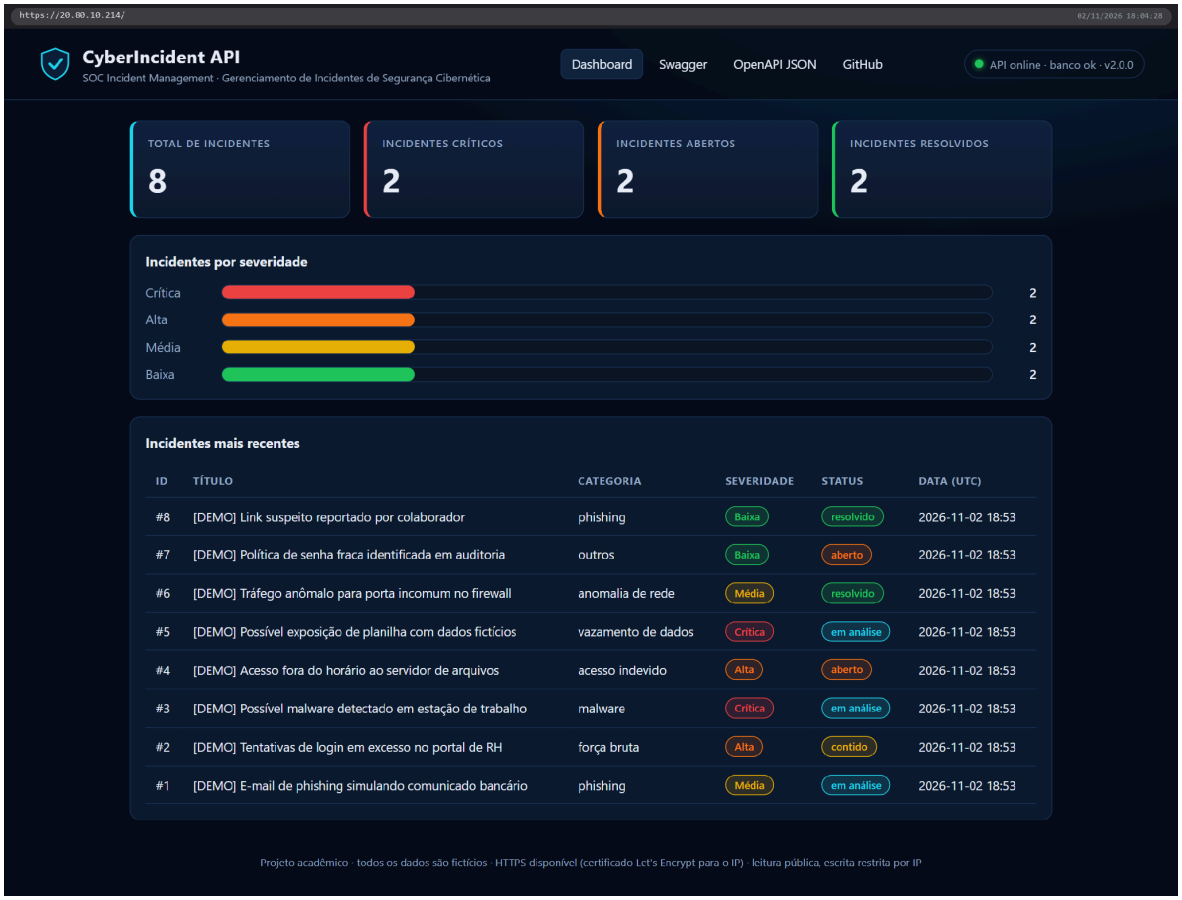


Figura 7 - Dashboard CyberIncident com indicadores e incidentes.

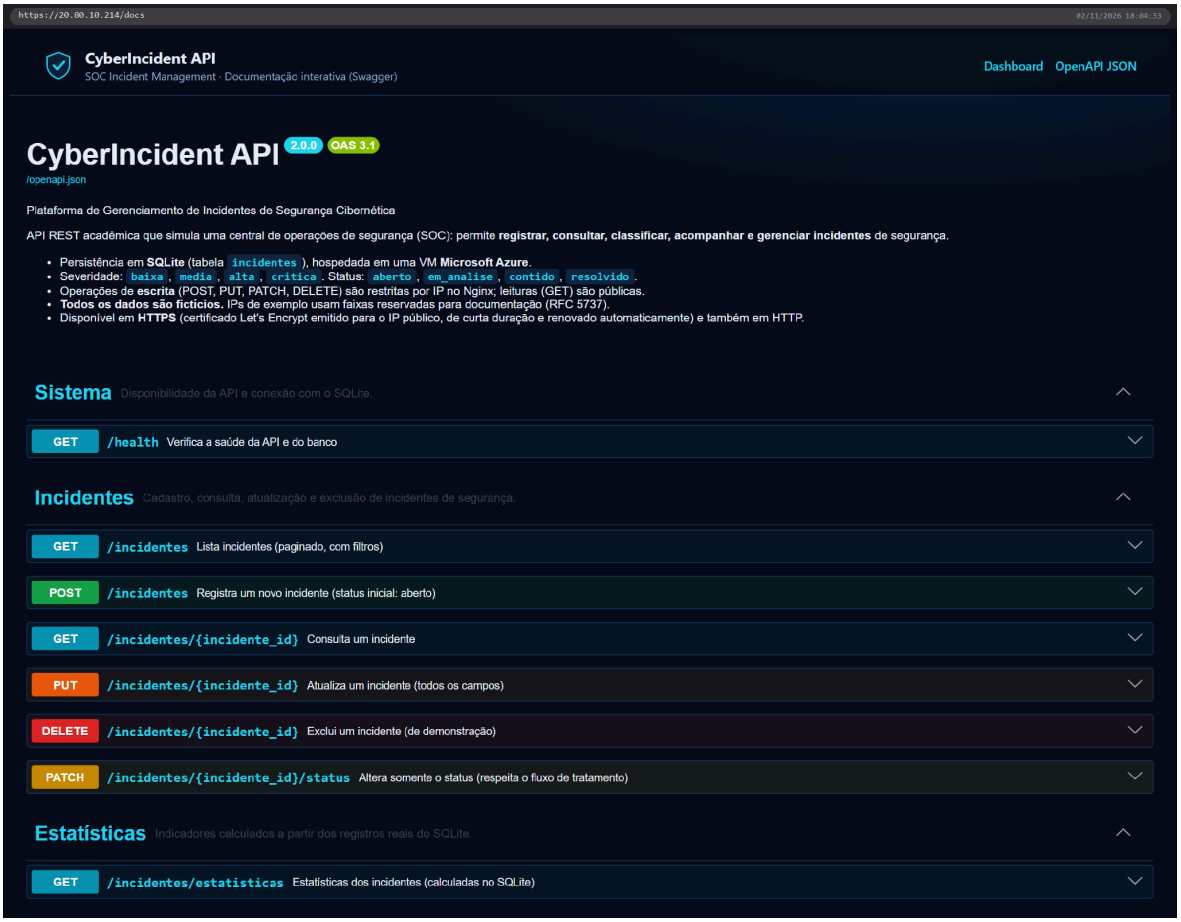


Figura 8 - Swagger personalizado da API publicada.

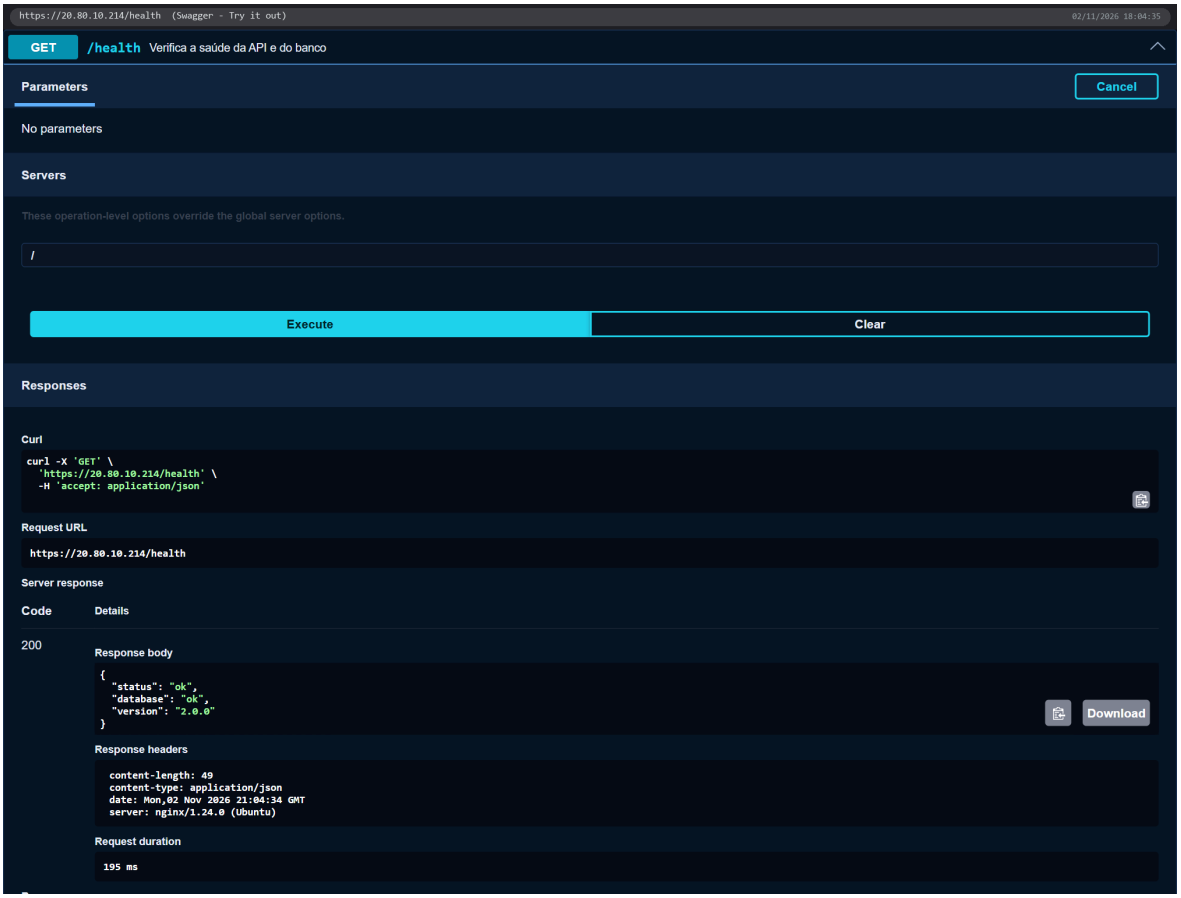


Figura 9 - Health check da API e do SQLite (HTTP 200).

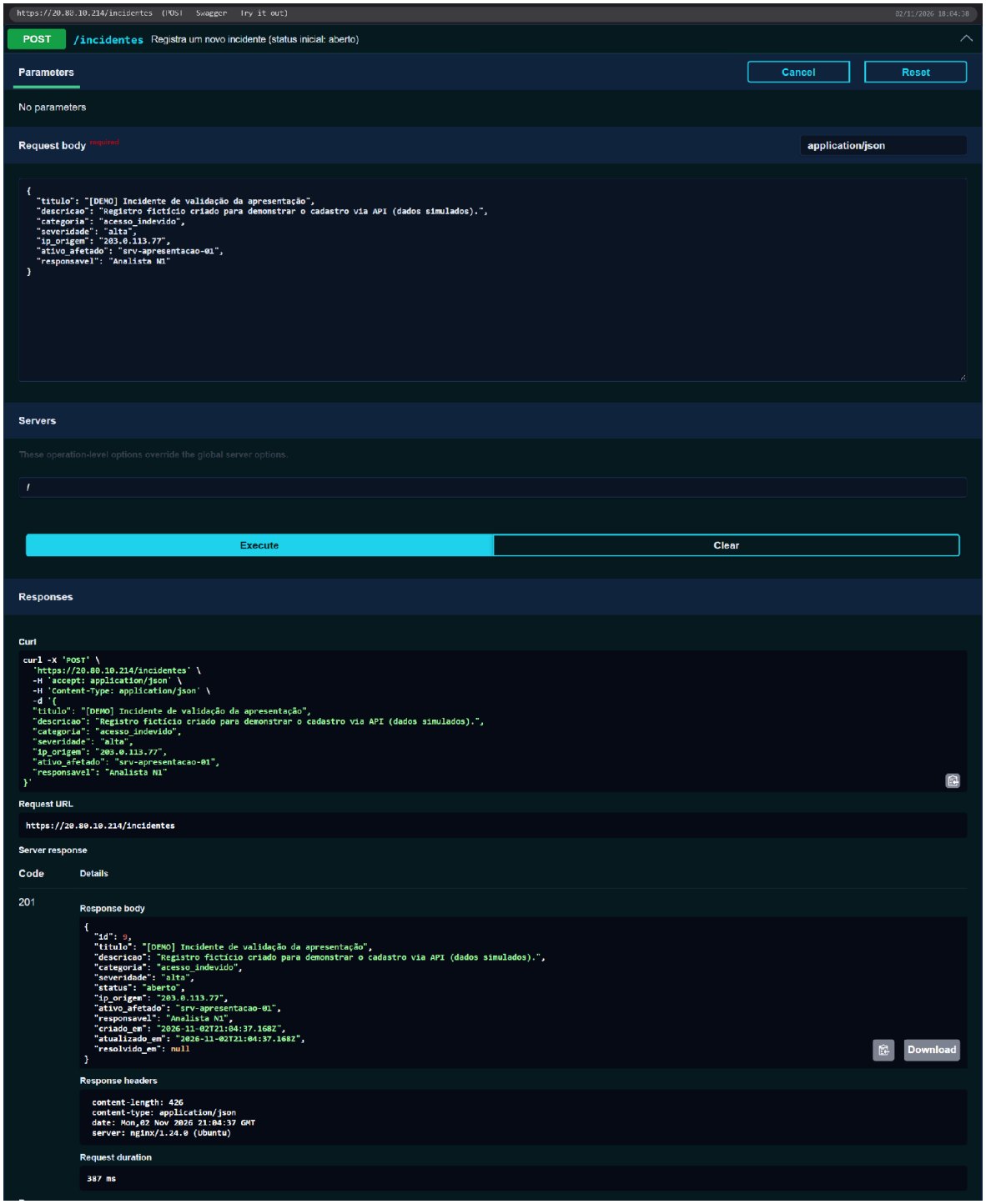


Figura 10 - Criação de incidente via POST (HTTP 201).

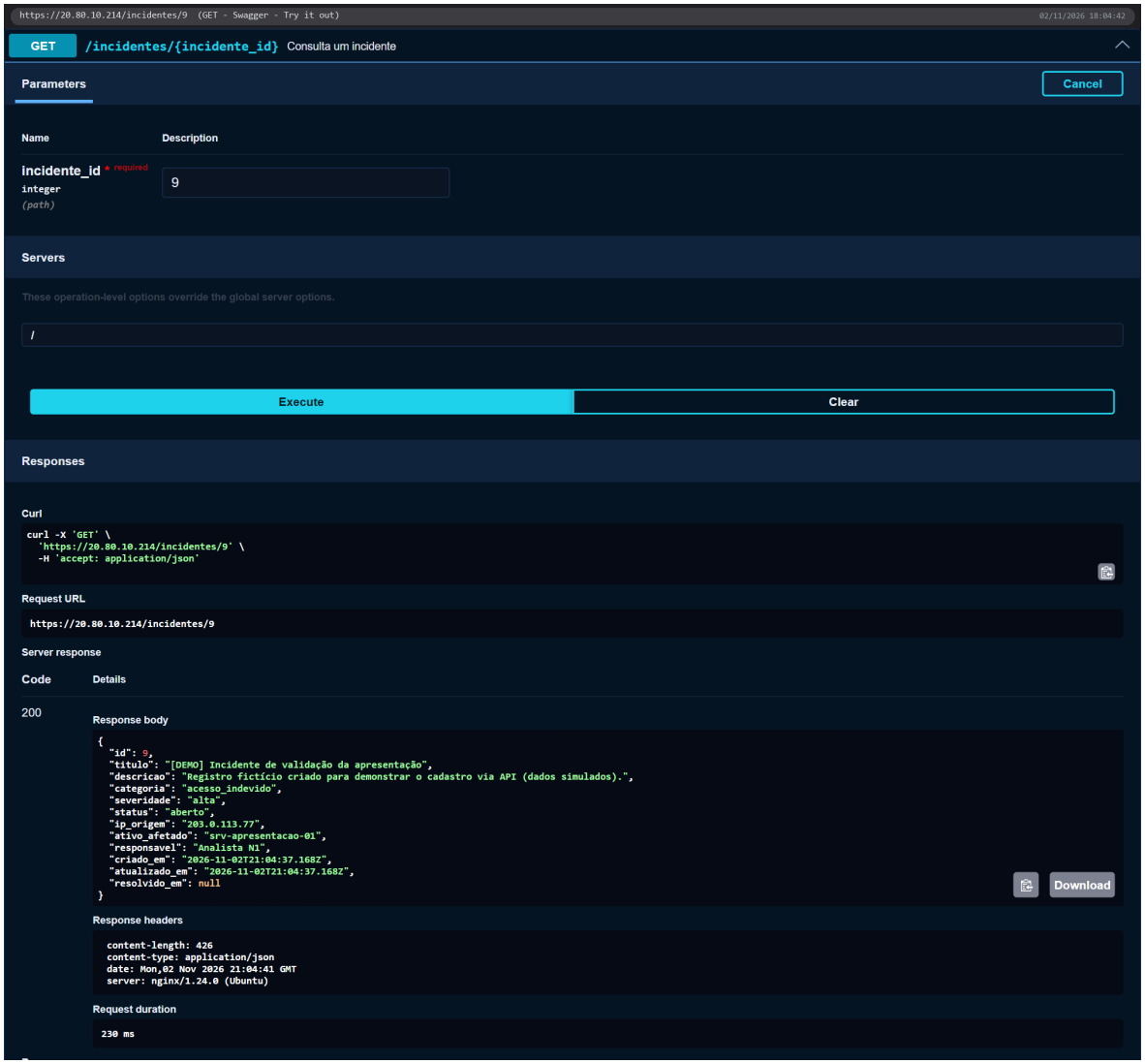


Figura 11 - Consulta de incidente via GET (HTTP 200).

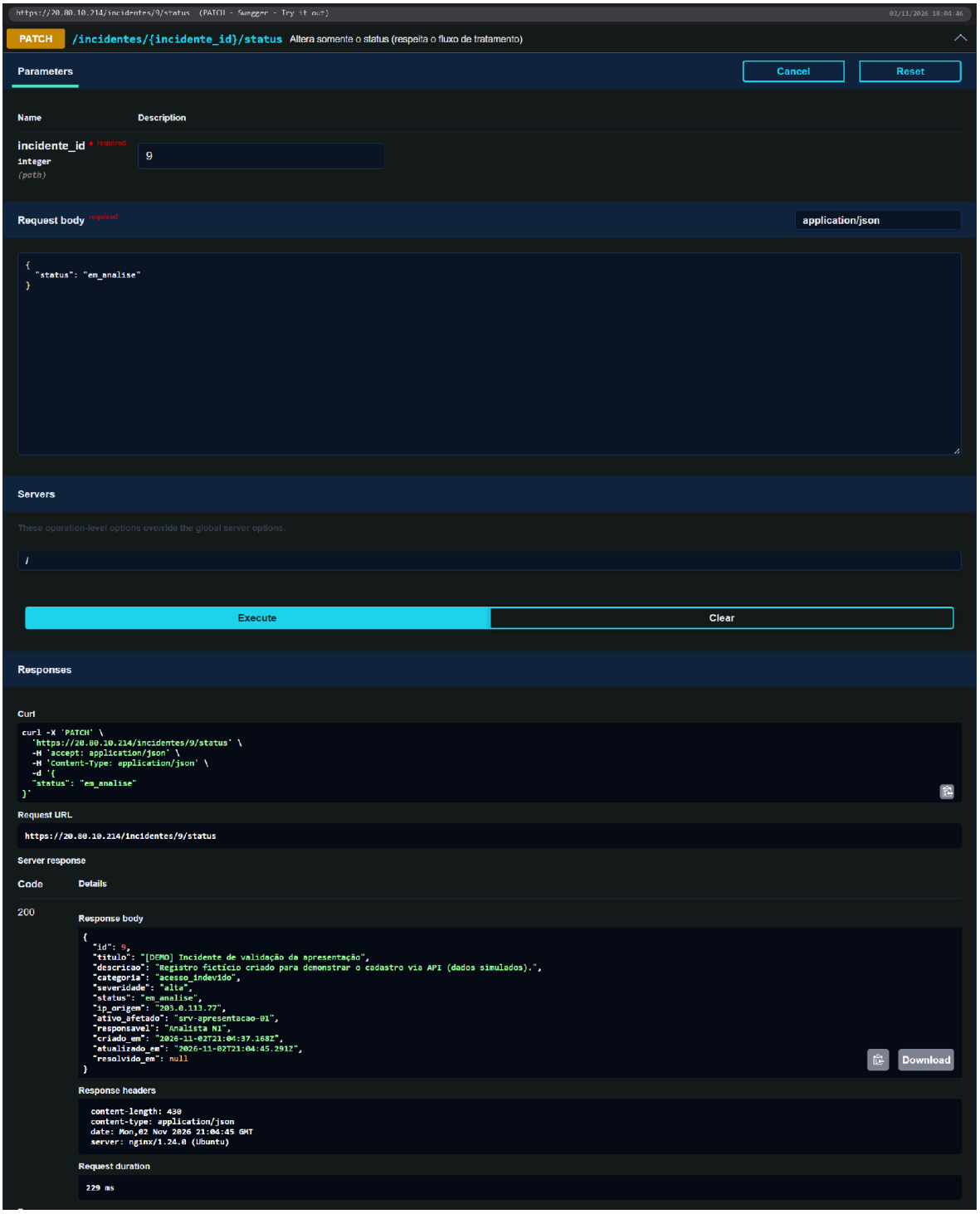


Figura 12 - Atualização de status via PATCH.

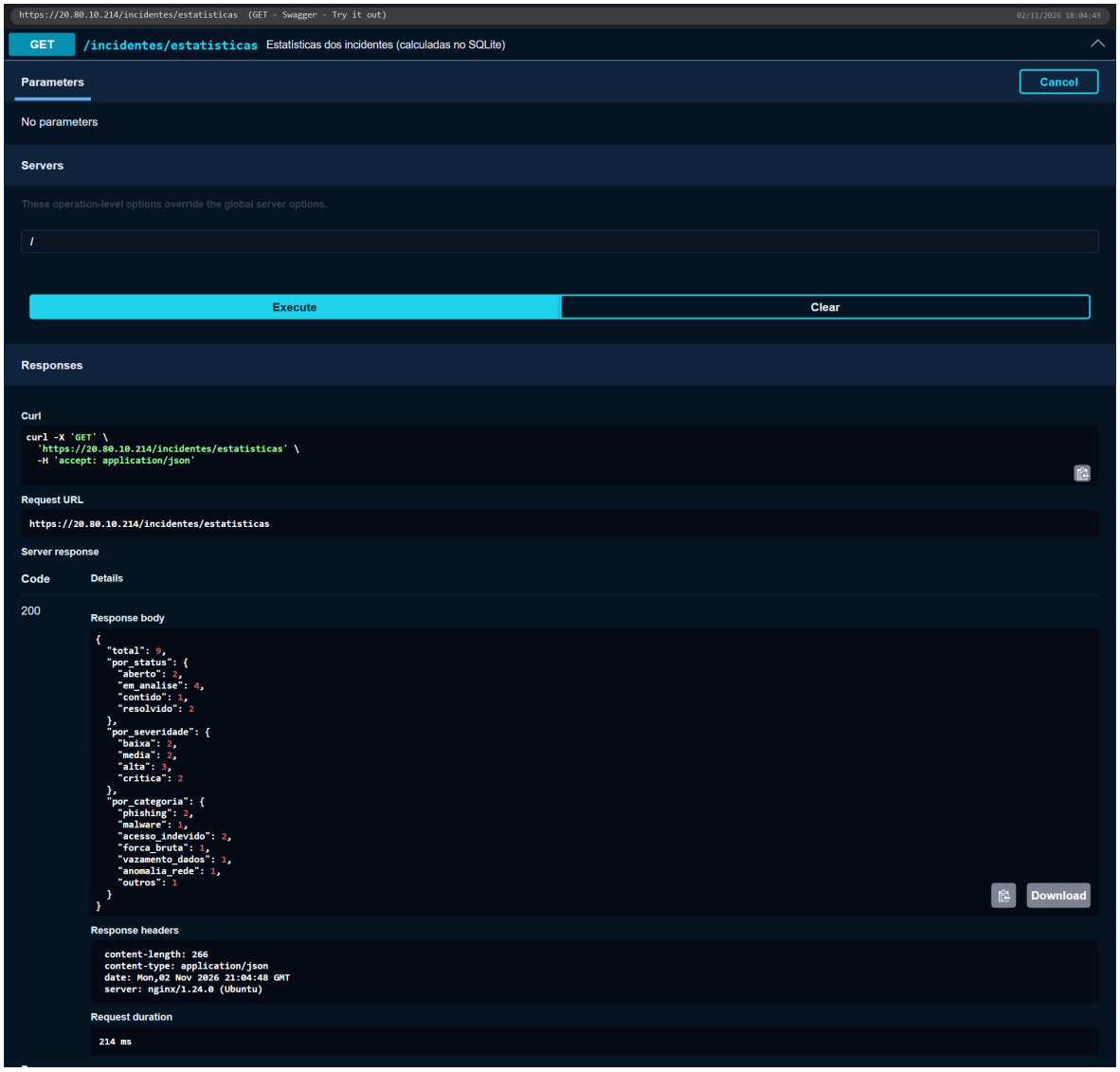


Figura 13 - Estatísticas geradas a partir do SQLite.

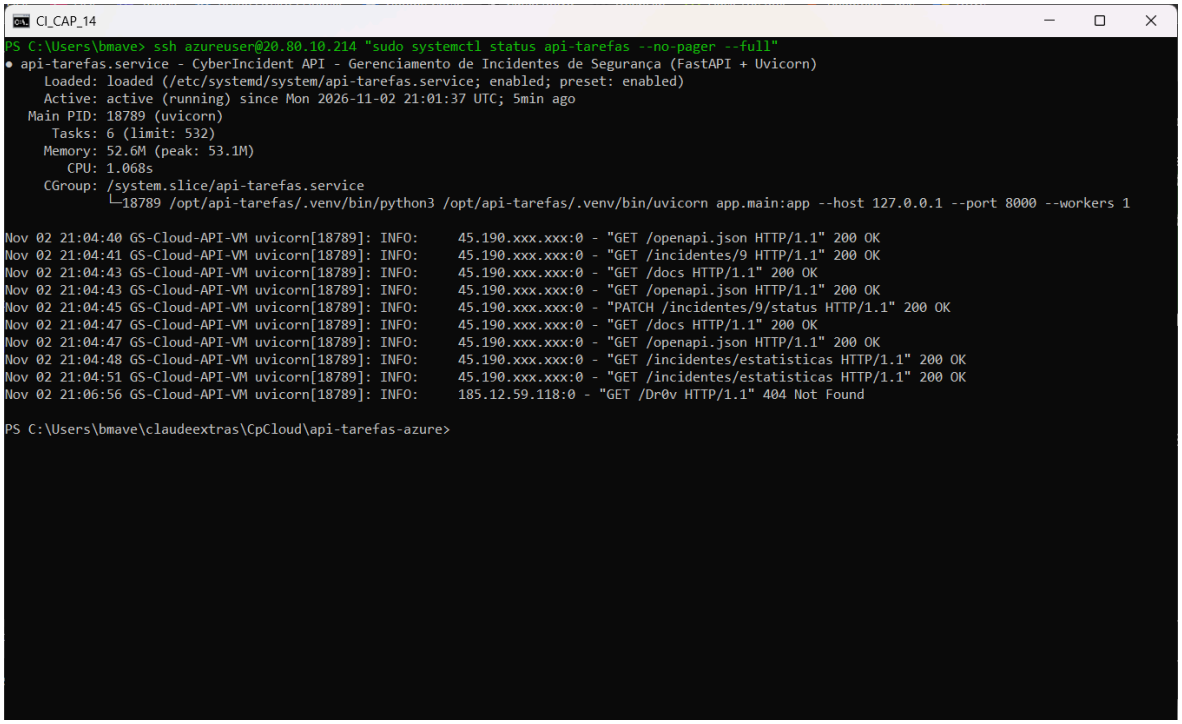


Figura 14 - Serviço systemd ativo na VM.

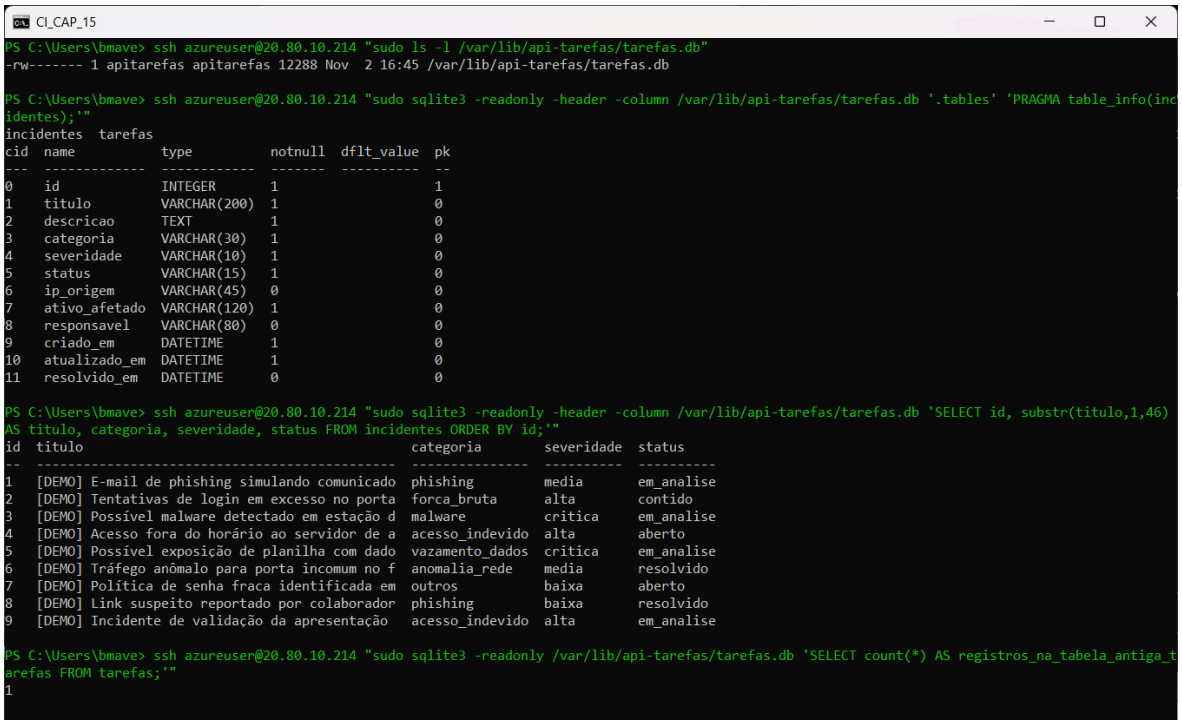


Figura 15 - Consulta direta aos registros SQLite.

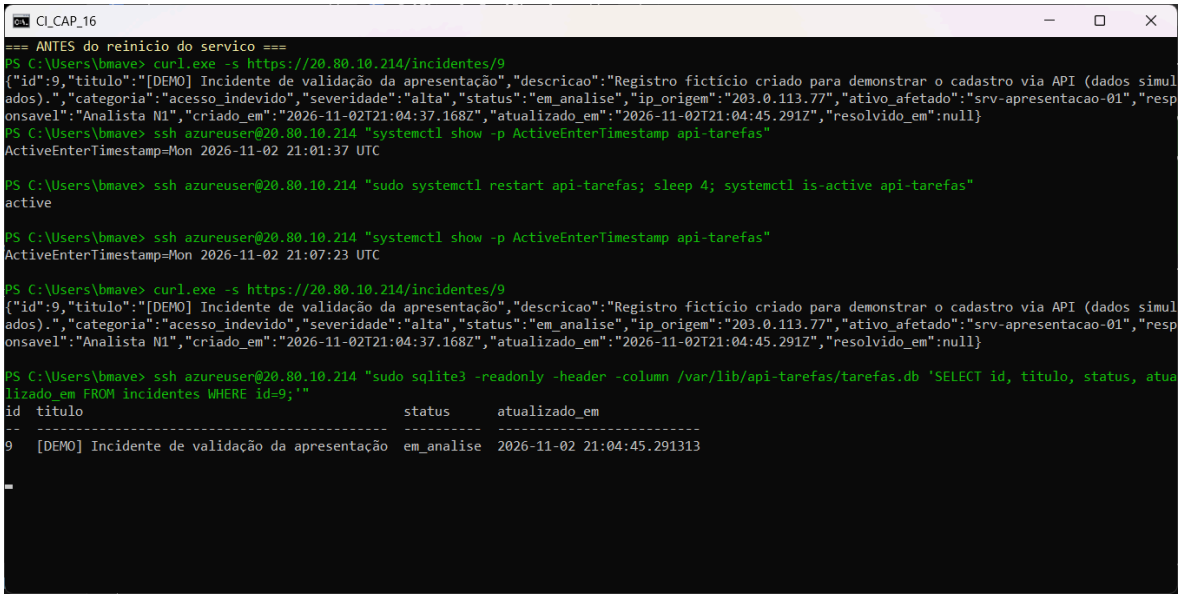


Figura 16 - Persistência após reinício do serviço.

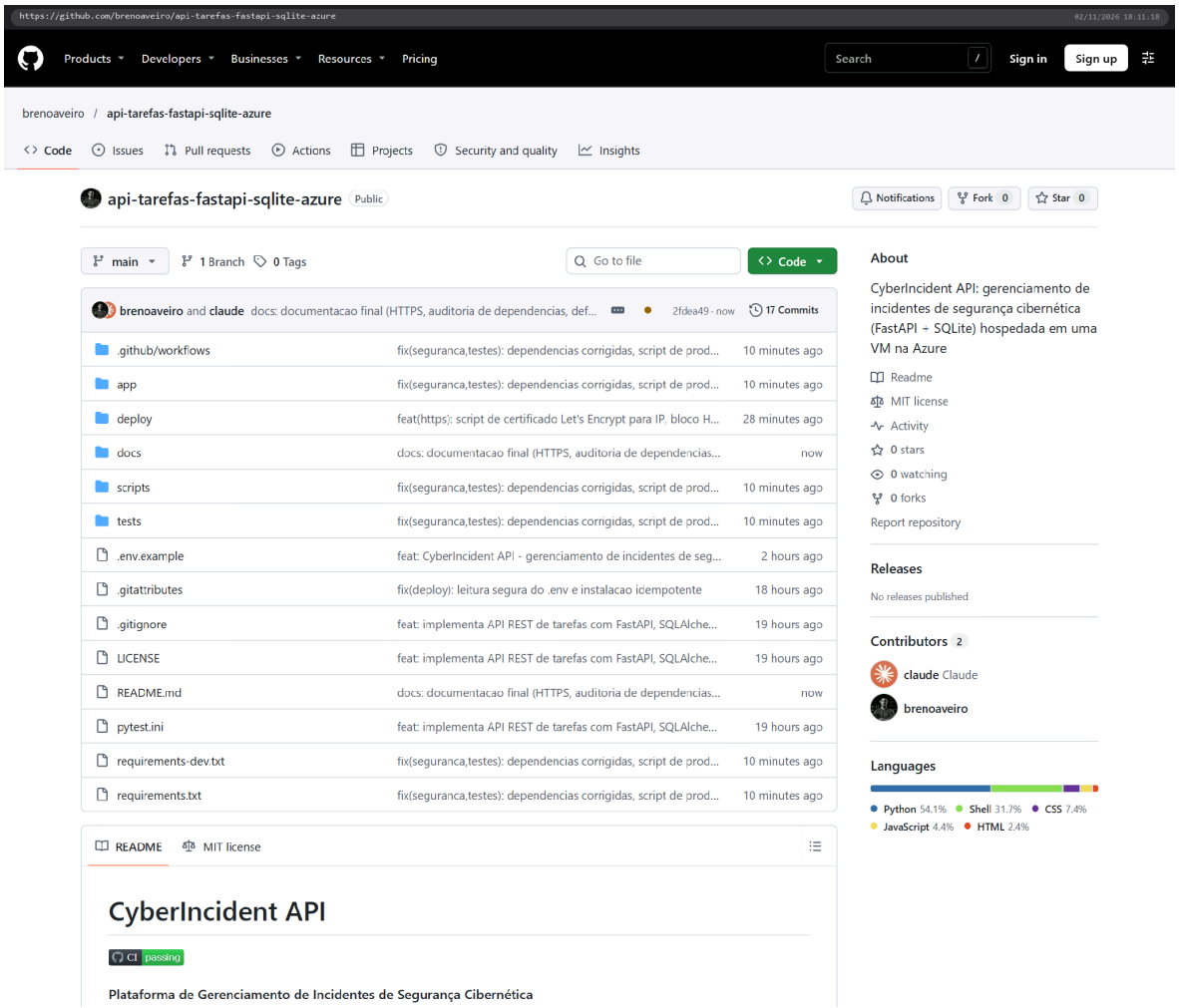


Figura 17 (parte 1) - Repositório e arquivos do GitHub.

CI passing

Plataforma de Gerenciamento de Incidentes de Segurança Cibernética

API REST acadêmica, em Python/FastAPI com SQLite, que simula o registro de incidentes de uma central de operações de segurança (SOC). Hospedada em uma máquina virtual na Microsoft Azure.

- **Aplicação (dashboard):** https://20.80.10.214/
- **Swagger (documentação interativa):** https://20.80.10.214/docs
- **Health check:** https://20.80.10.214/health
- Também disponível em HTTP: http://20.80.10.214/

> **Avisos:** todos os dados são **fictícios**. O HTTPS usa um certificado Let's Encrypt emitido para o **endereço IP** (validade de cerca de 6 dias, renovação automática); o HTTP continua ativo como contingência. A leitura (GET) é pública; a escrita (POST, PUT, PATCH, DELETE) é restrita por IP no Nginx, pois a API não tem autenticação de usuários.

O que a aplicação faz

Permite registrar, consultar, classificar, acompanhar e gerenciar incidentes como phishing, malware, força bruta, acesso indevido, vazamento de dados e anomalias de rede. Cada incidente tem severidade (`baixa` , `media` , `alta` , `critica`), status (`aberto` , `em_analise` , `contido` , `resolvido`), IP de origem opcional, ativo afetado e datas em UTC. A aplicação **não** detecta ameaças de verdade.

Método e rota	Função
`GET /`	Dashboard SOC (cartões, gráfico por severidade e tabela de incidentes)
`GET /health`	Disponibilidade da API e do SQLite
`GET /incidentes`	Lista com paginação e filtros (`severidade` , `status` , `categoria`)
`GET /incidentes/estatisticas`	Contagens reais por status, severidade e categoria
`GET /incidentes/{id}`	Consulta um incidente
`POST /incidentes`	Cadastra (status inicial `aberto`)
`PUT /incidentes/{id}`	Atualiza todos os campos
`PATCH /incidentes/{id}/status`	Altera somente o status (respeita o fluxo de tratamento)
`DELETE /incidentes/{id}`	Exclui um incidente de demonstração

O status segue o fluxo `aberto → em_analise → contido/resolvido` ; resolvido pode ser reaberto. Transições fora do fluxo retornam 422. `resolvido_em` é preenchido ao resolver e limpo ao reabrir.

Tecnologias

Python 3.12 · FastAPI 0.162 · Starlette 1.10 · Pydantic 2.14 · SQLAlchemy 2.0 · SQLite · Uvicorn · Nginx · systemd · Certbot (Let's Encrypt) · HTML/CSS/JavaScript sem frameworks · Swagger UI (servido localmente) · Pytest · GitHub Actions · Microsoft Azure (VM Ubuntu 24.04).

Estrutura do projeto

Figura 17 (parte 2) - README do repositório público.

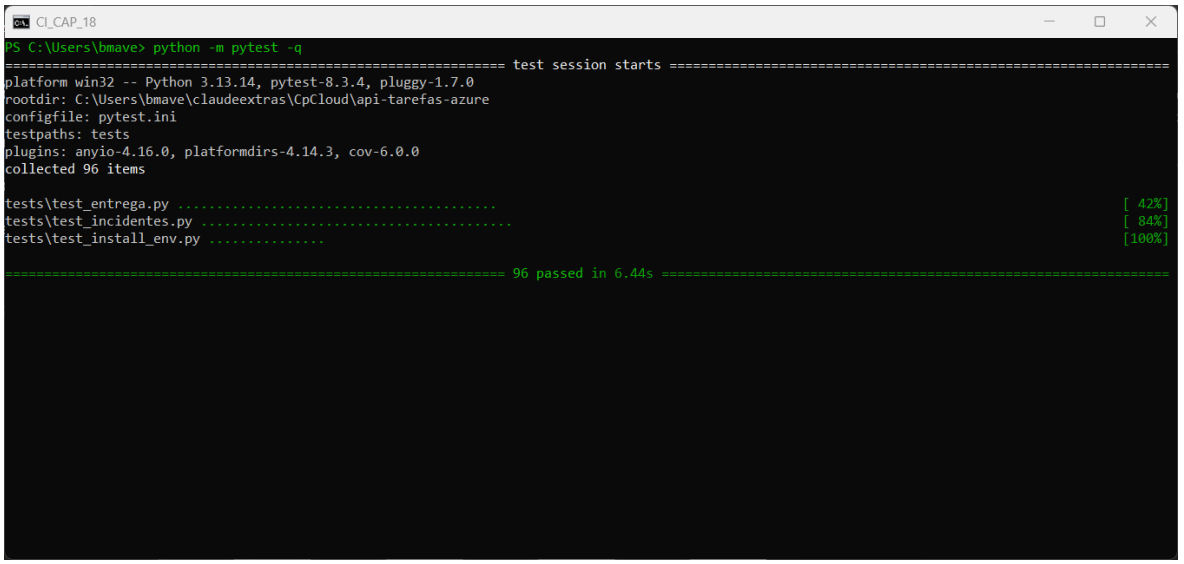


Figura 18 - Resultado dos testes automatizados.

12. Diagramas

Seis diagramas Mermaid documentam: (1) arquitetura Azure; (2) fluxo HTTP/HTTPS e Nginx; (3) modelo SQLite; (4) cadastro e estados de incidentes; (5) deploy GitHub/Azure; e (6) testes e validação. Os arquivos estão em docs/diagramas.

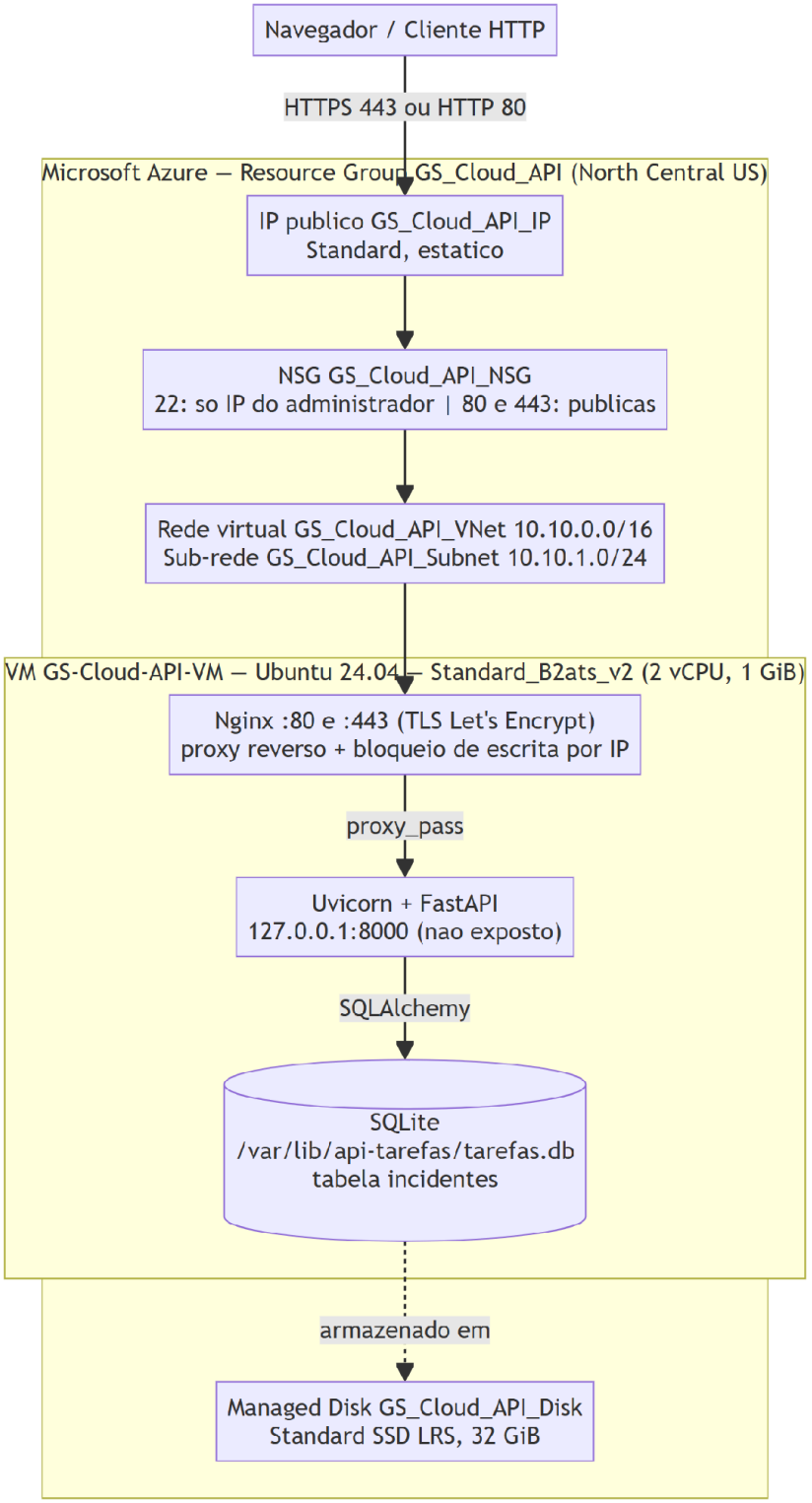


Diagrama 1. Arquitetura da CyberIncident API na Azure.

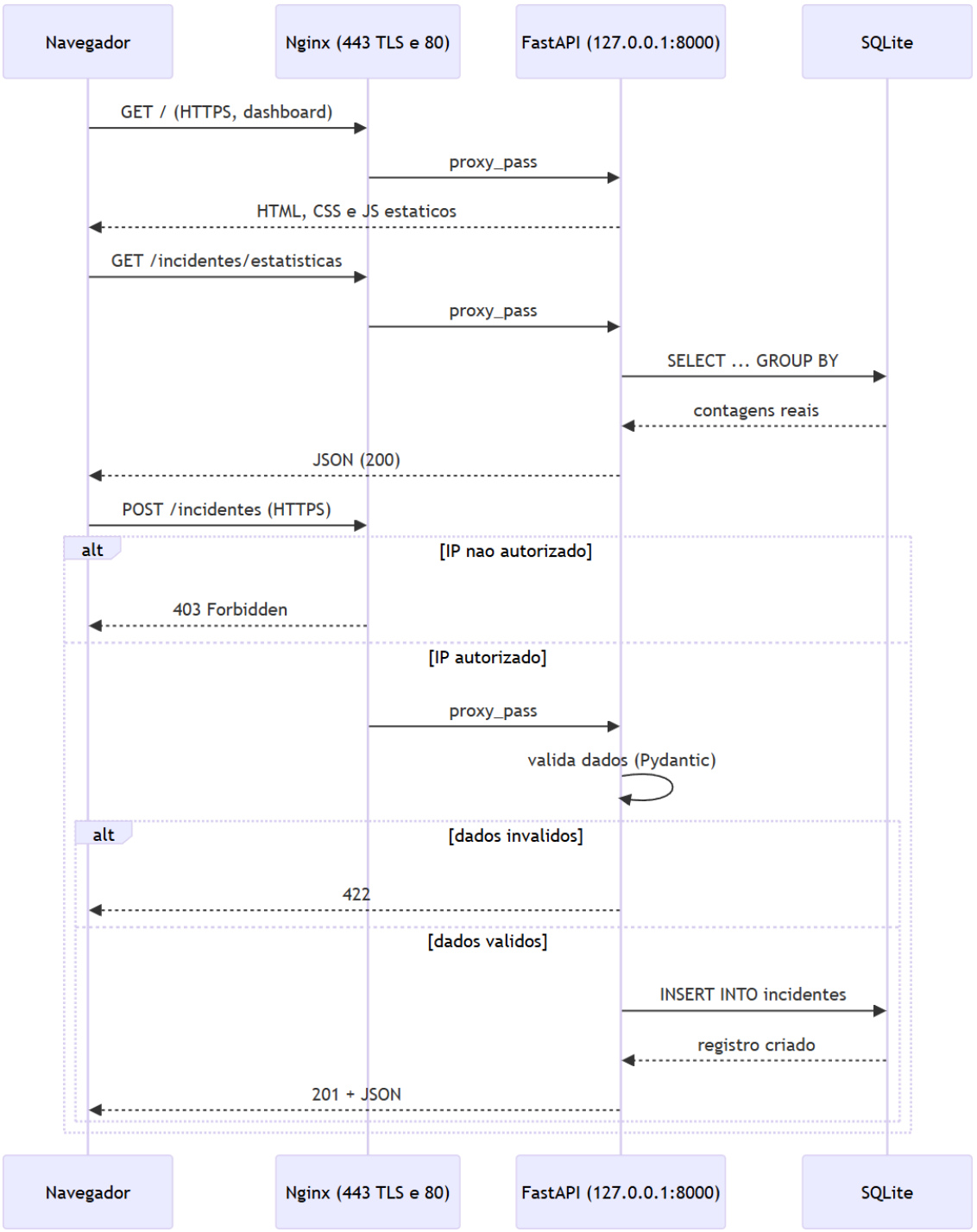


Diagrama 2. Fluxo HTTP/HTTPS entre os componentes.

INCIDENTES			
INTEGER	id	PK	
VARCHAR	titulo		
TEXT	descricao		
VARCHAR	categoria		phishing, malware, acesso_indevido, forca_bruta, vazamento_dados, anomalia_rede, outros
VARCHAR	severidade		baixa, media, alta, critica
VARCHAR	status		aberto, em_analise, contido, resolvido
VARCHAR	ip_origem		opcional, IPv4 ou IPv6
VARCHAR	ativo_afetado		
VARCHAR	responsavel		opcional
DATETIME	criado_em		UTC
DATETIME	atualizado_em		UTC
DATETIME	resolvido_em		preenchido ao resolver

Diagrama 3. Modelo de dados dos incidentes.

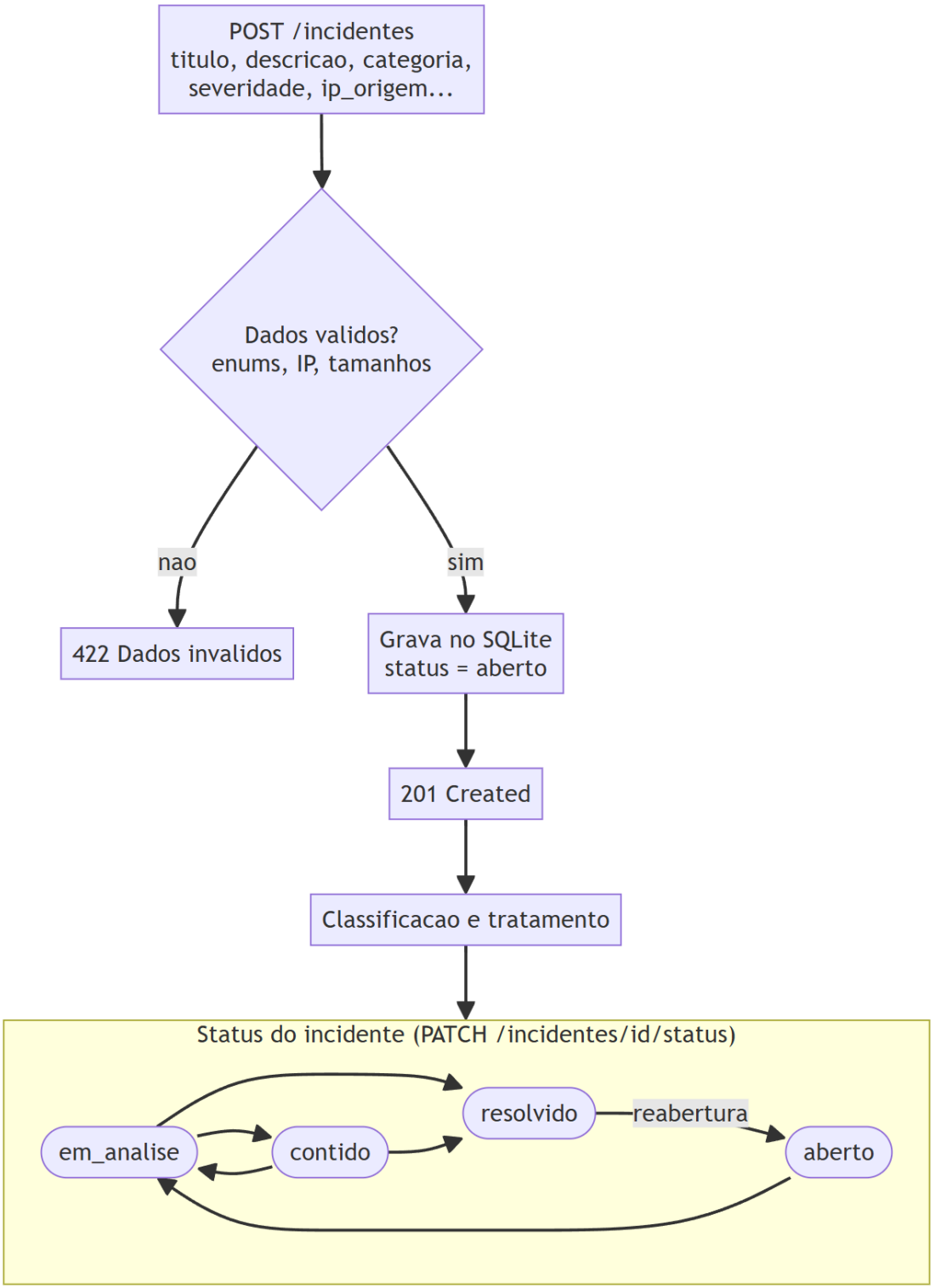


Diagrama 4. Cadastro e transições de status.

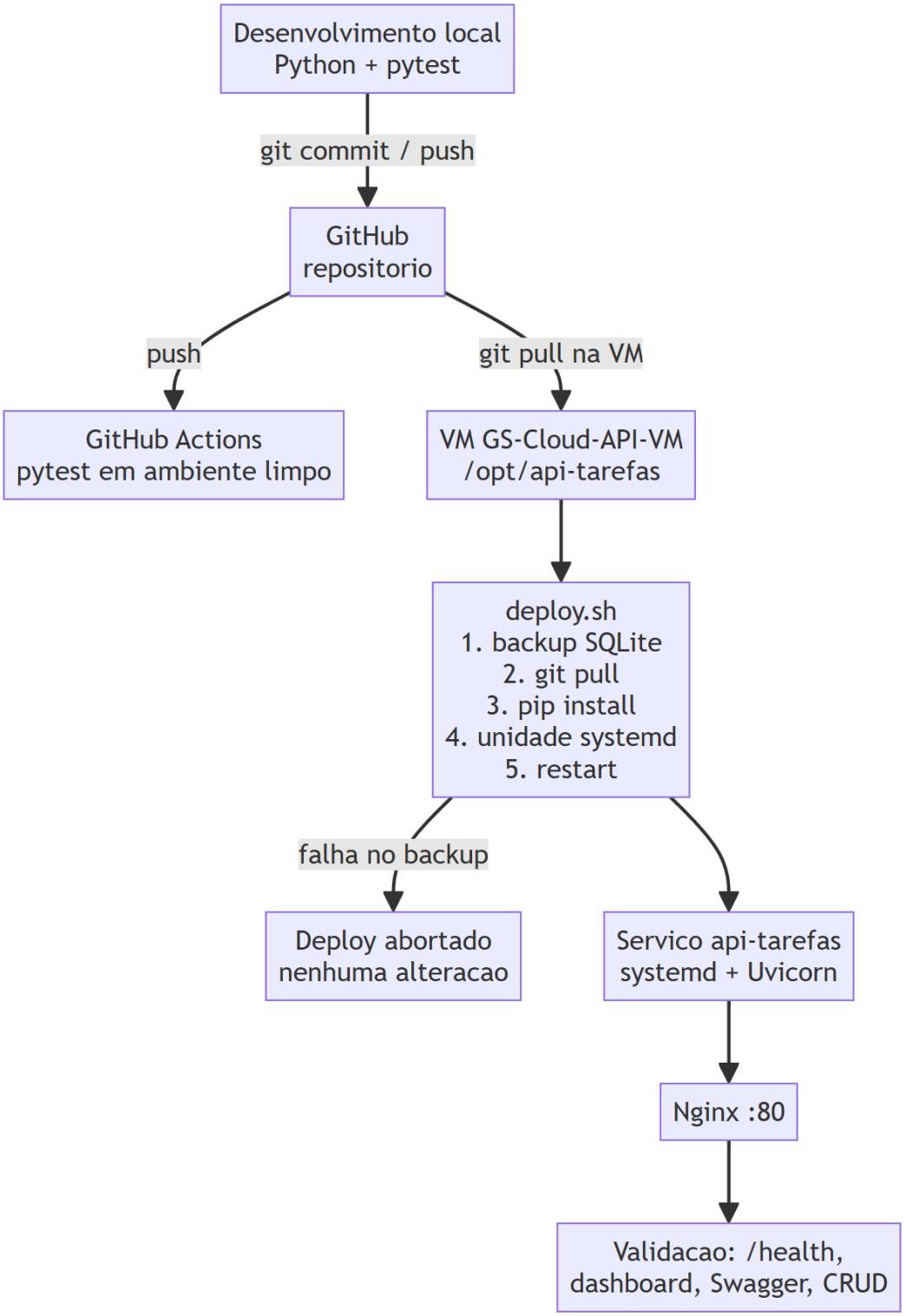


Diagrama 5. Implantação do GitHub na Azure.

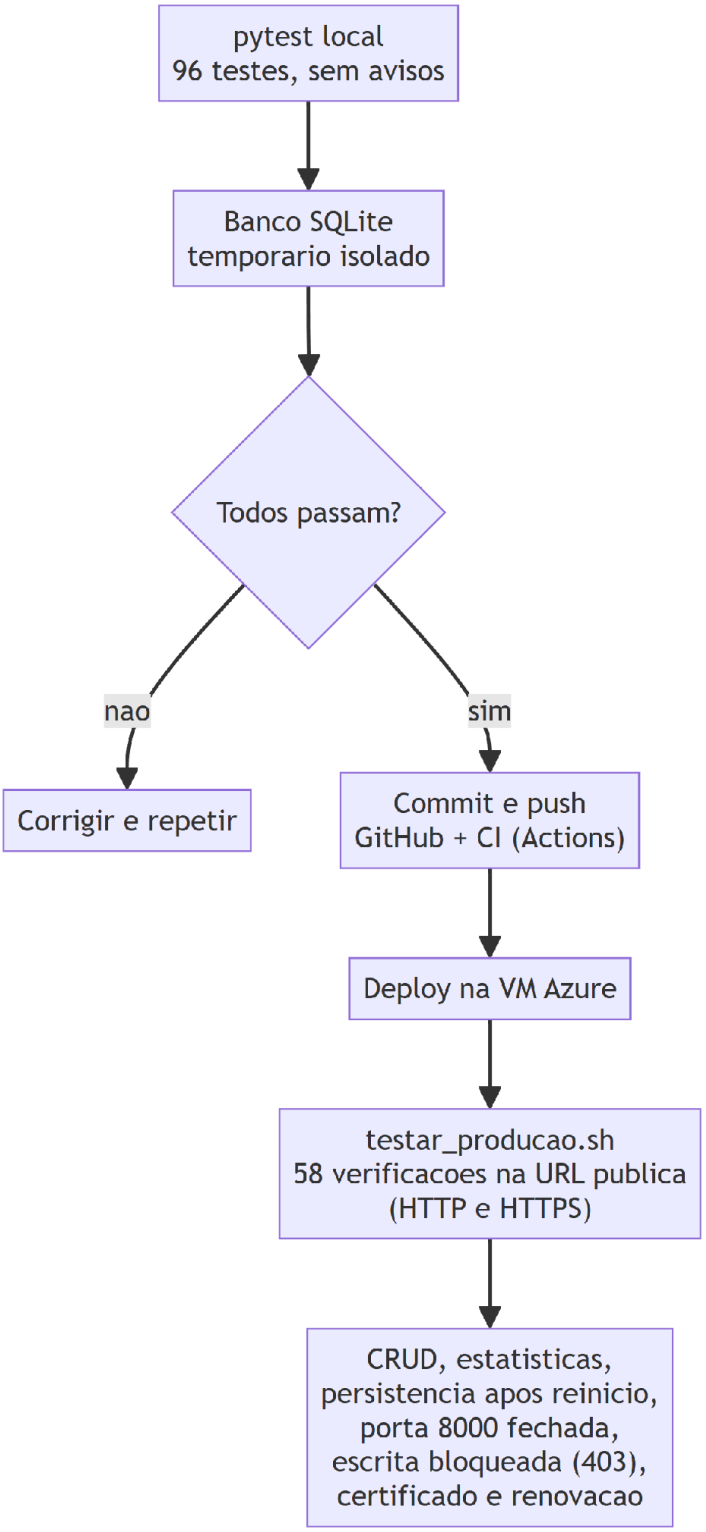


Diagrama 6. Etapas de teste e validação.

13. Custos

A tabela apresenta estimativas para Linux sob demanda, com a VM ligada 730 horas por mês. Não representa o gasto real nem o consumo dos créditos estudantis.

Recurso	Preço de tabela	Estimativa mensal

VM Standard_B2ats_v2 (North Central US)	US$ 0,0094 por hora	cerca de US$ 6,9
Disco Standard SSD de 32 GiB	cerca de US$ 2,64 por mês	cerca de US$ 2,6
IP público Standard	cerca de US$ 0,005 por hora	cerca de US$ 3,7
Certificado Let's Encrypt	gratuito	US$ 0
Total aproximado		**cerca de US$ 13 por mês**

VNet e NSG não têm tarifa direta. Desalocar a VM interrompe custos de computação, mas disco e IP continuam cobrados. O saldo deve ser consultado no Portal Azure.

14. Limitações e melhorias possíveis

- Sem autenticação individual. A escrita é restrita por IP; uma evolução pode usar token ou provedor de identidade sobre HTTPS.
- Certificado curto. Se a VM ficar desligada por vários dias, o certificado pode expirar até a próxima renovação. Uma alternativa é um domínio com certificado de maior duração.
- SQLite. Adequado para baixa concorrência; não oferece replicação nativa. Evolução: banco gerenciado.
- VM única de 1 GiB. Sem alta disponibilidade e com memória limitada. Evolução: mais memória e redundância.
- Backups locais. Estão no mesmo disco do banco. Evolução: armazená-los em local separado.
- Evidências Azure. As capturas do portal exigem acesso autenticado e não são geradas pelo CI.

15. Conclusão

A CyberIncident API foi publicada com dashboard e Swagger próprios, persistência SQLite e tabela legada preservada. Foram registrados 96 testes locais e 58 verificações de produção aprovados, além de CI, HTTPS com renovação automática e restrição de escrita por IP.

O trabalho aplicou conceitos de API REST, validação, testes e segurança, além de VM, SKU, rede e armazenamento Azure. As limitações e possíveis melhorias foram documentadas.

16. Referências

- FastAPI. Documentação oficial. https://fastapi.tiangolo.com
- Pydantic. Documentação oficial. https://docs.pydantic.dev
- SQLAlchemy. Documentação oficial. https://docs.sqlalchemy.org
- SQLite. Documentação oficial, incluindo o modo WAL e a API de backup. https://www.sqlite.org/docs.html
- Uvicorn. Documentação oficial. https://www.uvicorn.org
- Nginx. Documentação oficial. https://nginx.org/en/docs/
- systemd. Manual de referência. https://www.freedesktop.org/software/systemd/man/
- Microsoft. Tamanhos de máquinas virtuais no Azure. https://learn.microsoft.com/azure/virtual-machines/sizes

- Microsoft. Visão geral dos discos gerenciados do Azure. https://learn.microsoft.com/azure/virtual-machines/managed-disks-overview
- Microsoft. Grupos de segurança de rede do Azure. https://learn.microsoft.com/azure/virtual-network/network-security-groups-overview
- Microsoft. Calculadora de preços do Azure. https://azure.microsoft.com/pricing/calculator/
- Let’s Encrypt. Certificados de seis dias e para endereços IP: disponibilidade geral. https://letsencrypt.org/2026/01/15/6day-and-ip-general-availability
- Let's Encrypt. Certificados de curta duração e para IP no Certbot. https://letsencrypt.org/2026/03/11/shorter-certs-certbot
- Certbot. Guia do usuário. https://eff-certbot.readthedocs.io/en/stable/using.html
- GitHub. Documentação do GitHub Actions. https://docs.github.com/actions
- PyPA. Advisory Database e pip-audit. https://github.com/pypa/pip-audit
- IETF. RFC 5737: blocos de endereços IPv4 reservados para documentação. https://www.rfc-editor.org/rfc/rfc5737
- Swagger UI. https://swagger.io/tools/swagger-ui/